FAKULTI TEKNOLOGI KEJURUTERAAN ELEKTRIK DAN ELEKTRONIK

BVI 2135 EMBEDDED SYSTEM PROGRAMMING TOOL

AUTOMATIC WATER SPRINKLER AND MONITORING SYSTEM

BIL	NO ID	NAME
1.	VC23047	SYAMSUL ARIFIN BIN AWI BOWO
2.	VC23053	MUHAMMAD AMIRUL ADLY BIN MOHAMAD RASHIDI
3.	VC23054	AHMAD SYAHIDUL AMIN BIN MOHAMAD AZNAL

TABLE OF CONTENT

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

1.1 PROJECT BACKGROUND

Landscape maintenance at Universiti Malaysia Pahang Al-Sultan Abdullah (UMPSA) Pekan campus requires a reliable irrigation system to keep plants healthy and maintain the overall appearance of the campus. Areas such as gardens, roadside landscapes, and recreational zones need regular watering, especially due to Malaysia's tropical climate, which includes high temperatures, strong sunlight, and inconsistent rainfall. If irrigation is not managed properly, plants may suffer from poor growth, soil erosion can occur, and large amounts of water may be wasted.

Currently, many irrigation systems still depend on manual operation. In manual systems, maintenance staff must regularly check water tank levels and operate pumps based on experience. This method is time-consuming, inefficient, and highly dependent on human judgement. Problems such as delayed watering, overwatering, or insufficient watering may occur due to staff availability or misinterpretation of actual water needs. In addition, operating pumps without knowing the exact water level in the tank may cause dry running, which can damage the pump and reduce its lifespan.

With the development of embedded systems and Internet of Things (IoT) technology, irrigation systems can now be automated and monitored more effectively. IoT-based systems allow real-time data collection from sensors and enable users to monitor system conditions remotely through online platforms. This approach helps improve efficiency, reduce manual work, and support better water management by ensuring irrigation only takes place when sufficient water is available.

This project focuses on designing and developing a smart water sprinkling and tank monitoring system for the UMPSA Pekan campus. The system uses an ESP32 microcontroller, a waterproof ultrasonic sensor to measure water tank levels, and cloud-based data storage for monitoring purposes. In addition, a solar-powered energy system is integrated to support sustainable and energy-efficient operations. The proposed system aims to automate irrigation, improve monitoring accuracy, reduce water wastage, and minimize the need for manual supervision, contributing to the development of a smarter and more sustainable campus environment.

1.2 PROBLEM STATEMENT

At Universiti Malaysia Pahang Al-Sultan Abdullah (UMPSA), the existing irrigation system still relies heavily on conventional manual operation, which presents several operational and efficiency challenges. The activation and deactivation of water pumps and sprinklers depend entirely on human intervention, making the irrigation process inconsistent and highly dependent on staff availability. As a result, irrigation schedules may be missed or delayed, especially during weekends, public holidays, or unsuitable weather conditions. This manual dependency reduces the reliability of the irrigation system and increases the risk of inadequate water supply to plants.

Furthermore, the lack of automation contributes significantly to water wastage. Without real-time monitoring of water tank level, over-irrigation often occurs, leading to unnecessary water consumption. Conversely, under-irrigation may also happen when watering is insufficient or irregular, negatively affecting plant growth, landscape quality, and soil health. This inefficient water management contradicts sustainable resource usage practices and environmental conservation goals.

In addition, the current system poses a risk to irrigation equipment, particularly water pumps. Since there is no monitoring of tank water levels, pumps may continue operating even when the water supply is insufficient. This condition can cause pumps to run dry, resulting in overheating, mechanical wear, and potential motor burnout. Such equipment damage leads to increased maintenance costs, system downtime, and reduced operational lifespan of irrigation components.

Another major limitation of the conventional irrigation approach is the absence of monitoring and data logging capabilities. There are no historical records available for water usage, pump operating duration, or overall system performance. This lack of data makes it difficult for facility managers to evaluate irrigation efficiency, identify system faults, optimize watering schedules, or make informed decisions for future improvements. Therefore, an automated water sprinkler and tank monitoring system is necessary to address these issues by improving efficiency, reducing water wastage, protecting equipment, and providing reliable monitoring data.

1.3 OBJECTIVE

The objectives of this project are:

1. To design and implement an automated sprinkler control system using ESP32.
2. To develop a real-time water level monitoring system using a waterproof ultrasonic sensor.
3. To integrate cloud-based data storage and visualization.
4. To implement safety logic preventing pump operation during low-water conditions.
5. To produce a scalable and low-cost solution suitable for campus-wide deployment.

1.4 SCOPE & LIMITATION PROJECT

This project focuses on designing a smart water sprinkling and tank monitoring system with the following scope:

- Automatic control of water sprinkling based on predefined conditions
- Continuous monitoring of water tank level
- Wi-Fi based data transmission and cloud storage
- Local status display using OLED
- Bypass features to manually ON/OFF water pump

Despite the advantages, the system has certain limitations:

- System performance depends on Wi-Fi availability
- Sensor accuracy may be affected by environmental factors
- The system is designed for small-scale applications only

CHAPTER 2

LITERATURE

2.1 CONVENTIONAL METHODS ON MANUAL OBSERVATION

Conventional irrigation systems commonly rely on manual observation and human intervention to control water sprinklers and monitor water storage levels. In this approach, operators are required to visually inspect the water tank and manually switch irrigation pumps and sprinklers on or off based on experience or routine schedules. While this method is simple and low in initial cost, it is highly dependent on human availability and judgement, which can lead to inconsistent irrigation practices.

Manual observation often results in inefficient water management, especially when rainwater is used as the primary water source. Without continuous monitoring, operators may not accurately determine the actual water level inside the storage tank. This can cause the sprinkler system to operate when the tank water level is insufficient or remain inactive even when adequate water is available. Such conditions reduce the effective utilization of harvested rainwater and may disrupt regular irrigation activities.

Furthermore, conventional manual systems expose irrigation equipment to potential damage. Pumps may continue operating even when the tank is empty or at a critically low level, causing dry running that leads to overheating and mechanical failure. Since there is no automatic cutoff or alert mechanism, equipment damage is often only detected after a malfunction occurs, resulting in higher maintenance costs and system downtime.

Another major limitation of manual observation is the absence of data recording and performance monitoring. No historical information is available regarding water usage, pump operating duration, or system efficiency. This makes it difficult to analyze irrigation patterns, identify system faults, or implement improvements. Due to these limitations, conventional

manual irrigation methods are increasingly unsuitable for sustainable water management, highlighting the need for automated sprinkler and tank monitoring systems.

2.2 COMPONENT

This project integrates several hardware components that are widely used in modern IoT-based monitoring systems. Each component plays a crucial role in ensuring reliable operation and accurate data collection.

2.2.1 ESP32 Microcontroller

The ESP32 is a low-cost, low-power microcontroller series from Espressif Systems featuring integrated Wi-Fi and Bluetooth, making it ideal for IoT projects like smart home devices, wearables, and sensor networks, offering faster processing and more features than Arduinos while often using the familiar Arduino IDE for programming. As shown is Figure

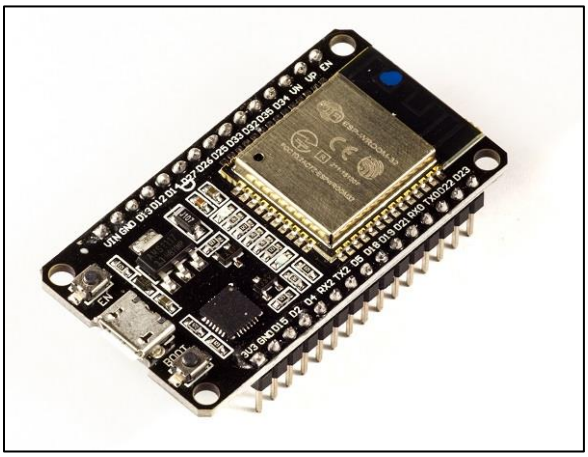


Figure 1: ESP32Module

This is the ESP32 Technical Specifications:

Table 1: ESP32 Specification

Parameter Name	Parameter Value
Microprocessor	
Operating Voltage	3.3V
DC Current on 3.3V Pin	50mA
DC Current on I/O Pin	40mA
Maximum Operating Frequency	240MHz

2.2.2 Ultrasonic Sensor (JSN-SR04T)

The JSN-SR04T ultrasonic sensor is designed for distance measurement in harsh environments. Unlike conventional ultrasonic sensors, it features a waterproof probe, making it suitable for water tank level monitoring. The sensor operates by emitting ultrasonic pulses and measuring the time taken for the echo to return after reflecting from the water surface. The measured time is converted into distance, which is then used to estimate the water level in the tank. As shown in Figure

Figure 2: Ultrasonic Sensor (JSN-SR04T)

Table 2: Ultrasonic Sensor (JSN-SR04T) Specification

Parameter Name	Parameter Value
Supply value	DC 3.0-5.5V
Working current	Max 8mA
Working frequency	40kHz
The longest range	600cm
Recent range	21cm
Long range accuracy	±1cm
Measuring angle	75 degrees
Connect	3-5.5V (power positive) Wiring mode Trig (control end) RX Echo (output terminal) TX GND (power negative)
Size	41*29*12mm
Operating temperature	-20℃—+70℃

2.2.3 OLED display 128x64 (SH110x)

The SH110x (commonly the SH1106 or SH1107) are popular single-chip CMOS OLED driver controllers used in small, low-power, and high-contrast monochrome displays for embedded systems like Arduino and Raspberry Pi projects. As Shown in Figure

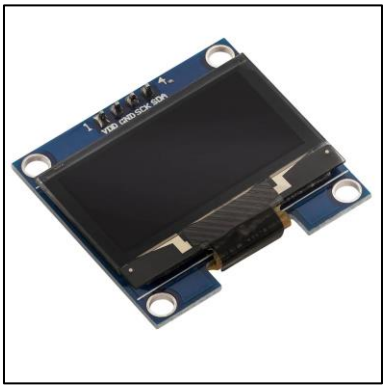


Figure 3: OLED Display 128x64

Table 3: OLED Display 128x64 Specification

Parameter Name	Parameter Value
Resolution	128x64 pixel
Display Type	Monochrome (one color only that usually white, blue, or yellow pixels on a black background)
Interface	I2C or SPI
Operating Voltage	3.3v or 5V
Power Consumption	very low power usage as the display does not require backlight

2.2.4 Relay Module (1 channel)

A 1-channel relay module is a compact electronic switch controlled by a low-voltage signal (like from an Arduino) that allows it to operate high-voltage, high-current devices (AC/DC motors, lights, etc.), featuring input pins (VCC, GND, Signal) and output screw terminals (COM, NO, NC) for isolation and simple switching in projects, often with status LEDs and optocouplers for safety. As shown in Figure

Figure 4: Relay Module (1 Channel)

This is the Relay Module Specification:

Table 4: Relay Module (1 Channel) Specification

Parameter Name	Parameter Value
	Digital
Rated Through Current	10A (NO), 5A (NC)
Control Signal	TTL Level
Max. Switching Voltage	250VAC/30VDC
Max. Switching Current	10A
Size	43mm x 17mm x 17mm

2.2.5 Giant Sun Controller (MPPT USB-PD)

The Giant Sun Controller, an MPPT USB-PD module, is used to efficiently manage power from a solar panel and battery system. This controller employs Maximum Power Point Tracking (MPPT) technology to maximize energy extraction from the solar panel by dynamically adjusting operating conditions. By ensuring optimal charging and power delivery, this component enhances system sustainability, enables off-grid operation, and reduces dependency on external power sources. As shown in Figure

Figure 5: Solar Controller

This is the Relay Module Specification:

Table 5: Solar Controller Specification

Parameter Name	Parameter Value
	Monocrystalline silicon
Power	50W / 100W
Open Circuit Voltage	12V
Short Circuit Current	2A
Solar Panel Size	42 x 28 cm
USB Output	5v / 2A
Protections	Over-voltage, low-voltage, reverse polarity, short-circuit, overload

2.2.6 Water Pump 12V

The 12V DC water pump is responsible for delivering water during the sprinkling process. It is activated based on predefined conditions such as sufficient water level in the tank. Proper control logic is implemented to prevent dry running, which could damage the pump.

Figure 6: Water Pump 12V

This is the Water Pump Specification:

Table 6: Water Pump 12V Specification

Parameter Name	Parameter Value
	Plastic
Voltage	12VDC
Maximum Rated Current	1000mA
Power	22W
Max Flow Rate	800 L/H
Max Circulating Water Temperature	100°C,
Inlet / Outlet	½” male thread

2.2.7 Signal Logic Level Converter

A Signal Logic Level Converter is a small circuit that enables communication between digital devices using different voltage levels (e.g., 3.3V and 5V) by safely stepping signals up or down, preventing damage and ensuring proper signal interpretation, commonly using MOSFETs for bi-directional translation in protocols like I2C, SPI, and UART. As shown in Figure

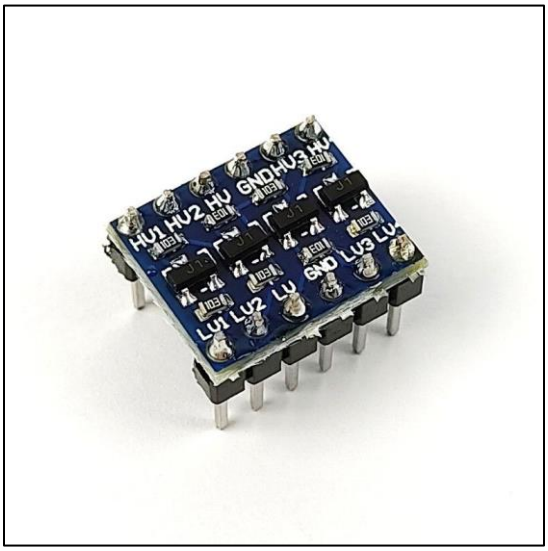


Figure 7: Logic Level Converter

This is the Level Converter Specification:

Table 7: Logic Level Converter Specification

Parameter Name	Parameter Value
	4 Channel
Mutual Transform	3.3V and 5V
Dimension	15.2mm x 12.7mm x 2.5mm
Weight	1.2g

2.3 PREVIOUS STUDIES

2.3.1 SMART IRRIGATION AND TANK MONITORING SYSTEM

Kumar Kunal, Md. Azhar Husain, N Srinivasan (2019). One related study titled *Smart Irrigation and Tank Monitoring System* introduced an automatic system that controls irrigation and monitors the water level in a tank. Sensors and microcontrollers are used to check the water level and control the irrigation process. The study showed that an automatic system can reduce manual work, save water, and prevent the pump from running without water.

The study focused on real-time water tank monitoring using Internet of Things (IoT) technology. In this system, water level data is collected continuously and sent online for monitoring. The researchers found that real-time monitoring helps users know the available water level and prevents the pump from operating when the tank is empty. This helps improve system reliability and reduces damage to the pump.

Other IoT-based irrigation systems have also been developed to improve water management. These systems usually use sensors, controllers, and wireless communication to automate watering and provide monitoring information. Although some systems use additional sensors such as soil moisture, the main idea of automatic control and water level monitoring is similar. These systems perform better than manual irrigation because they save water, reduce human effort, and are easier to manage.

2.3.2 WATER LEVEL CONTROL AND MONITORING SYSTEM

Petru Livinti, Marius V Tivlea, Popa Sorin Eugen (2023). In this work, the authors designed a low-cost system that monitors and controls the water level in a tank. The system uses an ultrasonic sensor (HC-SR04) to measure the water surface distance, a microcontroller board Arduino Uno drives a relay connected to a water pump to fill the tank, and another board NodeMCU ESP8266 sends water level data over Wi-Fi to an online platform (ThingSpeak), enabling remote monitoring.

According to their results, the system reliably stopped the pump automatically when the preset maximum water level was reached, preventing overflow and avoiding dry runs. Water level was measured and displayed via LCD, and real-time data was sent to the cloud for monitoring from anywhere.

The study showed that it is feasible to build an affordable, accurate, and automated tank water level control and monitoring system which reduces reliance on manual observation, helps protect pump equipment, and allows continuous data logging.

2.4 PROJECT THEORY

2.4.1 INTERNET OF THINGS CONCEPT

The Internet of Things (IoT) refers to the integration of physical devices such as sensors, controllers, and actuators with internet connectivity to enable data collection, monitoring, and control in real time. In this project, IoT is used to monitor the water level inside a rainwater storage tank and the operational status of the sprinkler motor. Through Wi-Fi communication, the controller transmits water level data and motor status to Google Sheets through an internet connection. This enables real-time data storage and access from any location. Wi-Fi communication supports continuous system monitoring and eliminates the need for wired data connections, making the system more flexible and scalable.

2.4.2 RAINWATER HARVESTING AND STORAGE SYSTEM

Rainwater harvesting is a sustainable method of collecting and storing rainwater for later use. In this system, rainwater is collected and stored in a water tank, which serves as the main water source for irrigation. Since rainwater availability depends on weather conditions, it is important to monitor the stored water level continuously. Proper monitoring ensures that the available rainwater is utilized efficiently while avoiding unnecessary water depletion.

2.4.3 WATER LEVEL MONITORING SYSTEM

Water level monitoring is a critical component of the proposed system. A water level sensor is used to measure the amount of water available in the tank. The sensor continuously provides water level data to the controller. This information is used to determine whether sufficient water is available for irrigation. By monitoring the water level, the system can prevent the motor from operating when the tank water is below a safe level, thereby protecting the pump from dry running and potential damage.

2.4.4 AUTOMATIC SPRINKLER CONTROL SYSTEM WITH SAFETY FEATURE

The automatic sprinkler system is designed to irrigate plants without manual intervention by operating according to a predefined schedule. In this project, the sprinkler is programmed to activate automatically three times a day at predetermined times. The controller manages the motor and sprinkler operation while simultaneously enforcing safety conditions to ensure reliable performance. Before each scheduled irrigation cycle, the system checks the current water level in the rainwater storage tank. If the water level is below the minimum threshold, the motor will not be activated even when the scheduled time is reached. This integrated control mechanism improves irrigation consistency, reduces dependency on human operation, prevents pump damage caused by dry running, and ensures that watering only occurs when sufficient water is available.

2.4.5 DATA LOGING AND VISULIZATION SYSTEM

Data logging is implemented by storing water level and motor status data in Google Sheets. This data is then visualized using Looker Studio, which provides real-time dashboards displaying water level status and motor operation. The visualization allows users to easily understand the current system condition, track changes over time, and detect abnormal behavior. This feature enhances system transparency and supports better decision-making and maintenance planning.

CHAPTER 3

METHODOLOGY

3.1 PROJECT DESIGN

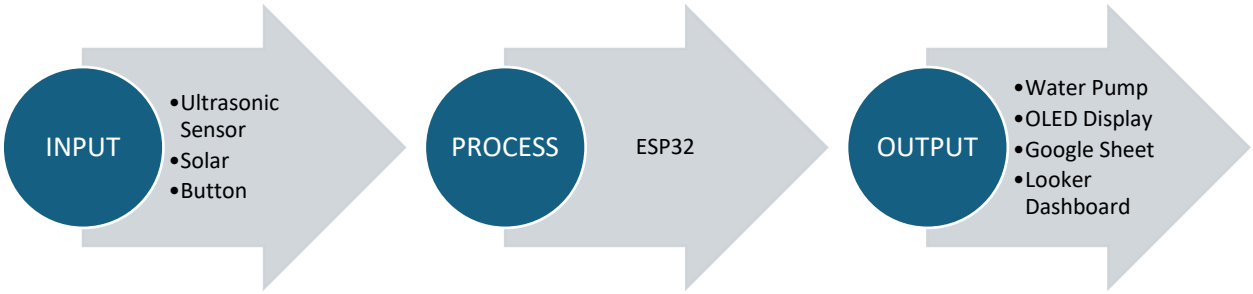


Figure 8: Project Design

3.1.1 INPUT

The input of the system consists of a water level sensor, button and a power supply unit. An ultrasonic sensor is used to measure the water level inside the rainwater tank by detecting the distance between the sensor and the water surface. This information is used to determine whether there is enough water for sprinkler operation. A solar panel is used to supply power to the system. The electrical energy generated by the solar panel is stored in a battery, which provides a 12V power supply to the water pump. The battery allows the system to operate even when there is no sunlight and makes the system portable, enabling it to function in locations without a fixed power source. The button allow bypass to manually run the water pump without waiting the schedule for maintenance and ease of troubleshooting.

3.1.2 PROCESS

The process stage is handled by the ESP32 microcontroller, which acts as the central control unit of the system. The ESP32 receives water level data from the ultrasonic sensor and processes it according to predefined control logic. Based on this logic, the controller determines whether sufficient water is available in the tank before activating the sprinkler system. The ESP32 also manages the automatic sprinkler schedule, ensuring the system operates three times daily at preset times while applying safety checks. It also can overwrite the operation to manually run water pump receiving signal from button. In addition, the ESP32 uses Wi-Fi connectivity to transmit real-time water level data and motor status to Google Sheets, enabling remote monitoring and data storage. This processing stage integrates sensing, decision-making, automation, and communication on a single platform.

3.1.3 OUTPUT

The output stage consists of system actions and information displays generated based on the processing results. The water pump is activated or deactivated automatically to control the sprinkler system according to the schedule and water level conditions also force run the water pump outside to its schedule by button. An OLED display provides local, real-time visual feedback by showing the current water level and motor status. The motor status will display in two way such as Auto Run and Manual Run according to how the water pump triggered. Simultaneously, system data is sent to Google Sheets for data logging purposes. This data is further visualized through a Looker Studio dashboard, which presents real-time information on water level and motor status in an easy-to-understand graphical format. These outputs allow both local and remote monitoring, improving system transparency, reliability, and ease of management.

3.1.4 CIRCUIT DIAGRAM

Here are our circuit diagram design, it shows the overview of the component connection consist of Solar Panel, Giant Sun Controller, Battery Storage, AC to DC 5v, ESP32, JSN-SR04T Ultrasonic Sensor, 5v to 3.3v Logic Level Converter, Relay and 12V Water Pump.

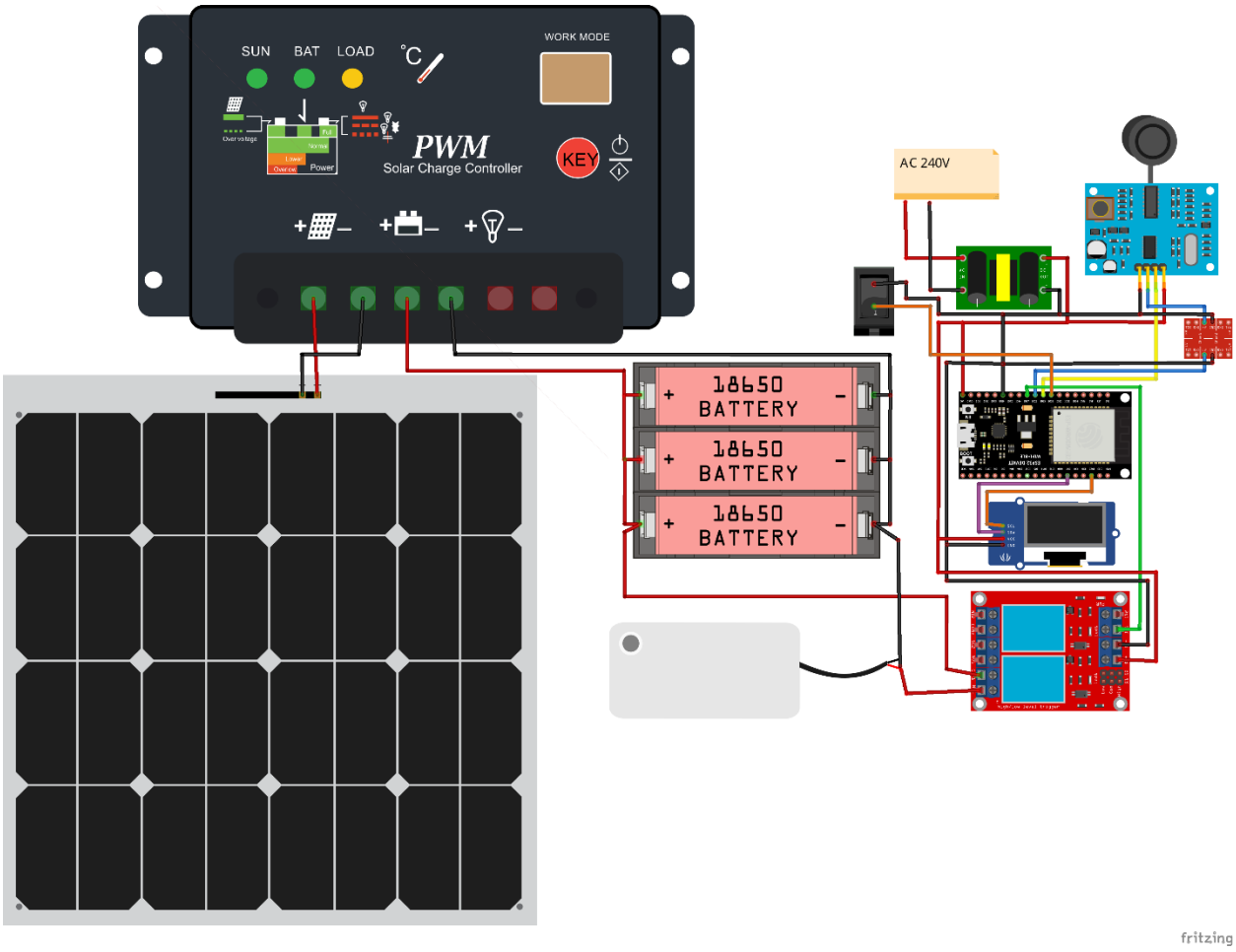


Figure 9: Circuit Diagram

3.1.5 AIR VENTILATION SENSOR HOLDER

Here are the 3D CAD design of the Air Ventilation Sensor Holder, this part used for holding the sensor inside the tank and to give a proper ventilation to ensure the water inside the tank have pressure and easy to flow to the water pump.

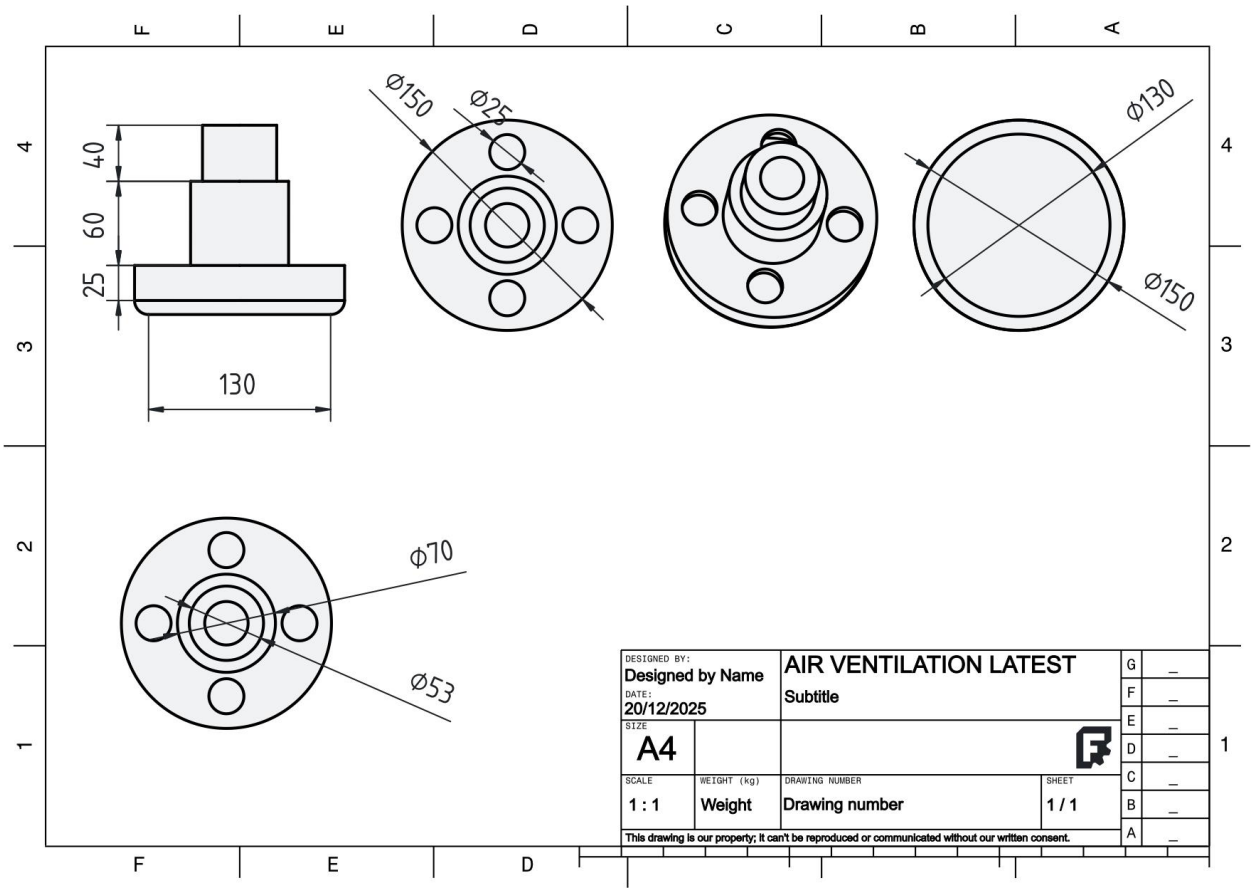


Figure 10: Air Ventilation part design

3.2 PROJECT DEVELOPMENT & IMPLEMENTATION

In the development and implementation of this project, they have software development, hardware development, data log and visualization in the project.

3.2.1 SOFTWARE DEVELOPMENT & IMPLEMENTATION

Here is the software implementation that we make our system software; the development process consists of importing required libraries, input and output setup, HTTP API setup for the Google Sheet, logic control of the system also local information visualization.

3.2.1.1 Arduino Code for Smart Water Sprinkling System

The Arduino code is implemented on an ESP32 microcontroller to control a water sprinkling system and monitor a water tank. The system integrates ultrasonic sensing, relay control, OLED display, time scheduling, and data transmission to Google Sheets.

Key Libraries Used as shown in Figure:

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>
```

Figure 11: Libraries Setup

Wi-Fi Connection Setup:

The ESP32 connects to the specified Wi-Fi network using the `ssid` and `password`. The connection is continuously retried until successful. As shown in Figure

```
// ===================== WIFI ======================
const char* ssid = "MAABMR";
const char* password = "Yoil21203";
```

Figure 12: Wi-Fi Setup Connection

OLED Display Initialization:

The OLED (128×64) is initialized via the I2C bus (SDA=21, SCL=22). Base on the Figure

```
// ================ OLED SH110X ====================
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SH1106G display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
```

Figure 13: OLED Setup

Ultrasonic Sensor and Relay Pins:

- TRIG and ECHO pins are configured to measure distance using the ultrasonic sensor.
- RELAY pin controls the water pump.
- Button pin are configured to operate Motor pump Manually

Base on The Figure

```
// ================== Ultrasonic Pins ==============
#define TRIG 26
#define ECHO 25

// ================== Relay Pin ====================
#define RELAY 27

// ================== Manual Button ================
#define BUTTON_PIN 33   // Button to GND (INPUT_PULLUP)
```

Figure 14: Ultrasonic & Relay pin Configuration

NTP Time Client:

The NTPClient library fetches Malaysia local time (UTC+8), which allows scheduled watering. As shown in Figure

```
// =============== NTP Time (Malaysia UTC+8) ========
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 8 * 3600, 60000);
```

Figure 15: Setup Malaysia Time

Water Level Measurement:

The Ultrasonic sensor measurement the distance from the sensor to the water surface as shown in Figure

```
// ================ Read Ultrasonic Sensor ===========
float readDistance() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  long duration = pulseIn(ECHO, HIGH, 30000); // timeout 30ms
  if (duration == 0) return tankHeight;

  float distance = duration * 0.034 / 2;
  return distance;
}
```

Figure 16: Water Level Settings

Watering Control:

The water pump is activated via the relay for 1 minute when triggered by scheduling watering as shown in Figure

```
// ================= Water 1 minute =================
void waterForOneMinute() {
  Serial.println("Motor ON (watering 1 minute)");
  digitalWrite(RELAY, HIGH);
  delay(60000);
  digitalWrite(RELAY, LOW);
  Serial.println("Motor OFF");
}
```

Figure 17: Motor Pump Settings

Scheduled Watering:

The System is programmed to water at three specific times Morning, Evening, and Night. As Shown in Figure

```
// ===== Scheduled watering =====
if (hour == 6 && minute == 00 && !waterMorning && waterLevel > lowWaterCutoff) {
  waterForOneMinute();
  motorStatus = "ON";
  waterMorning = true;
}

if (hour == 12 && minute == 00 && !waterEvening && waterLevel > lowWaterCutoff) {
  waterForOneMinute();
  motorStatus = "ON";
  waterEvening = true;
}

if (hour == 20 && minute == 00 && !waterNight && waterLevel > lowWaterCutoff) {
  waterForOneMinute();
  motorStatus = "ON";
  waterNight = true;
}
```

Figure 18: Scheduling Sprinkler Settings

Manual On Water Pump:

Manual button has highest priority, turning the motor ON while pressed, otherwise allowing AUTO schedule to run, and turning the motor OFF when neither is active.

```
// ================ MANUAL CONTROL ================
manualMode = (digitalRead(BUTTON_PIN) == LOW);
if (manualMode) {
  digitalWrite(RELAY, HIGH);
  motorStatusText = "MANUAL ON";
  motorStatusValue = 1;
  autoRunning = false; // stop AUTO if manual pressed
} else {
  // If AUTO running, keep it ON
  if (autoRunning) {
    digitalWrite(RELAY, HIGH);
    motorStatusText = "AUTO ON";
    motorStatusValue = 1;
  } else {
    digitalWrite(RELAY, LOW);
    motorStatusText = "OFF";
    motorStatusValue = 0;
  }
}
```

Figure 19: Manually Operate Motor Pump using Button

OLED Display Update:

OLED Display Time, Motor Status, and Water Level. Besides that, OLED updates every 15 minutes. As shown in Figure

```
// =============== OLED UPDATE ======================
void updateOLED(String timeStr, float waterLevel, String motorStatus) {
  display.clearDisplay();

  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Water Tank Monitor");

  display.drawLine(0, 10, 127, 10, SH110X_WHITE);

  display.setCursor(0, 15);
  display.print("Time: ");
  display.println(timeStr);

  display.setCursor(0, 30);
  display.print("Motor: ");
  display.println(motorStatus);

  display.setCursor(0, 45);
  display.print("Level: ");

  display.setTextSize(1);
  display.setCursor(55, 42);
  display.print(waterLevel, 1);
  display.print("cm");

  display.display();
}
```

Figure 20: OLED Display Settings

Data Transmission to Google Sheets:

Sends Sensor Reading, Motor Status, and Timestamp in JSON format to a Google App Script URL. As Shown in Figure

```
// =============== Send to Google Sheets =============
void sendToSheets(String timestamp, float distance, float waterLevel, String motorStatus) {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin(scriptURL);
    http.addHeader("Content-Type", "application/json");

    String json = "{";
    json += "\"Timestamp\":\"" + timestamp + "\",";
    json += "\"Distance\":" + String(distance, 2) + ",";
    json += "\"Water Level\":" + String(waterLevel, 2) + ",";
    json += "\"Motor Status\":\"" + motorStatus + "\"";
    json += "}";
```

Figure 21: Settings to Send data to Google Sheet

3.2.1.2 Google Apps Script for Google Sheet Integration

The Google Apps Script acts as a web application endpoint to receive JSON data from the ESP32 and log it in to Google Sheets.

HTTP Handling Function:

Handles POST requests send by the ESP32 and Parses incoming JSON data and insert it into the Google Sheet as shown in Figure

```
function doGet(e){
  return ContentService.createTextOutput("Web App is running and ready for POST requests.");
}

function doPost(e) {
  try {
    var ss = SpreadsheetApp.getActiveSpreadsheet();
    var sheetName = "Data";

    // Check if sheet exists, create if not
    var sheet = ss.getSheetByName(sheetName);
    if (!sheet) {
      sheet = ss.insertSheet(sheetName);
    }
```

Figure 22: Confirm JSON data from ESP32

Sheet Management:

Check if the sheet “Data” exists and ensure header row consistency with [“Timestamp”,”Distance”,”Water Level”,”Motor Status”,”ID”]. As shown in Figure

```
// ====== AUTO-HEADER PROTECTION ======
var headers = ["Timestamp","Distance","Water Level","Motor Status","ID"];

// If header row empty OR not match → rewrite header
var firstRow = sheet.getRange(1, 1, 1, headers.length).getValues()[0];
var headerMissing = false;

for (var i = 0; i < headers.length; i++) {
  if (firstRow[i] !== headers[i]) {
    headerMissing = true;
    break;
  }
}
```

Figure 23: Auto Header Settings

Inserting a New Data Row

Each row in the google sheet come out with new data from ESP 32, by appending rows the system builds a historical record for analysis and visualization as shown in Figure

```
// Insert new data row
sheet.appendRow([
  new Date(),                 // Timestamp
  data.Distance,              // Distance
  data["Water Level"],        // Water Level
  data["Motor Status"],       // Motor Status
  newID                       // Auto ID
]);
```

Figure 24: Insert Data According to Specific Columns

3.2.2 HARDWARE DEVELOPMENT STEP

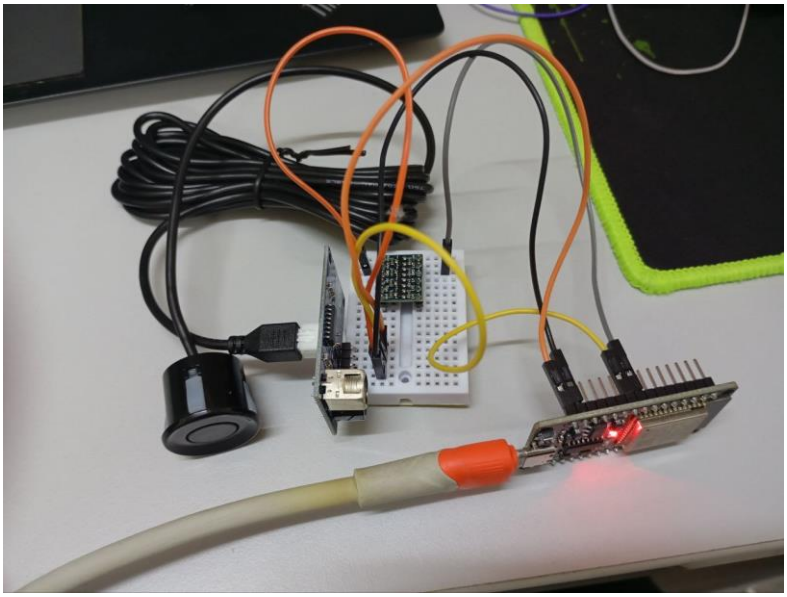
Here are the step by step how we develop our system hardware; the development process consists of component wiring, pin connection and enclosure box assembly.

Table 8: Hardware Development Step

<table>
<tr><th>NO</th><th>WORK STEP</th><th>ATTACHMENTS</th></tr>
<tr><td>1.</td><td>Connect the Ultrasonic sensor to the ESP32

Pin Connection:

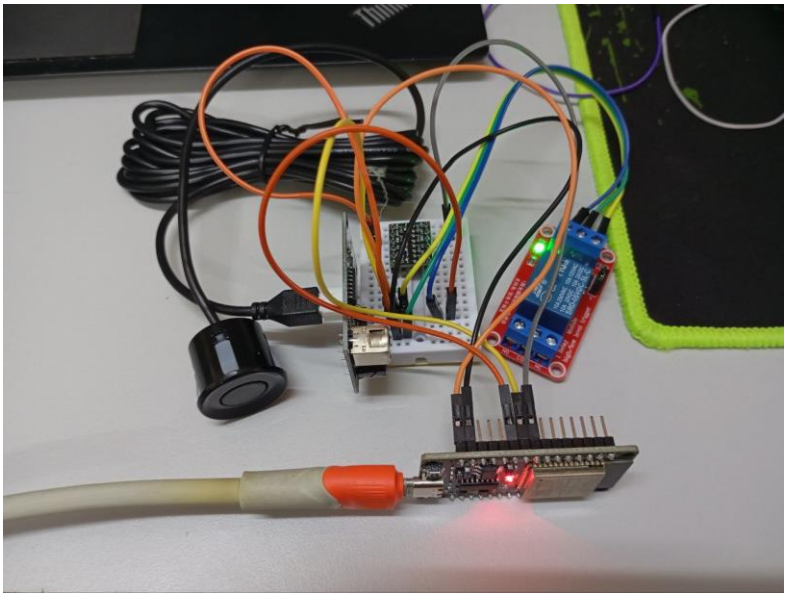
<table>
<tr><th>Ultrasonic</th><th>ESP32</th></tr>
<tr><td>VCC</td><td>VIN</td></tr>
<tr><td>TRIG</td><td>D25</td></tr>
<tr><td>ECHO</td><td>D26</td></tr>
<tr><td>GND</td><td>GND</td></tr>
</table>

The ECHO pin connected to D26 via Logic Level Converter that step down the operating voltage from 5v to 3.3v allowing a stable signal</td><td>  </td></tr>
</table>

2. Connect Relay to ESP32

Pin Connection:

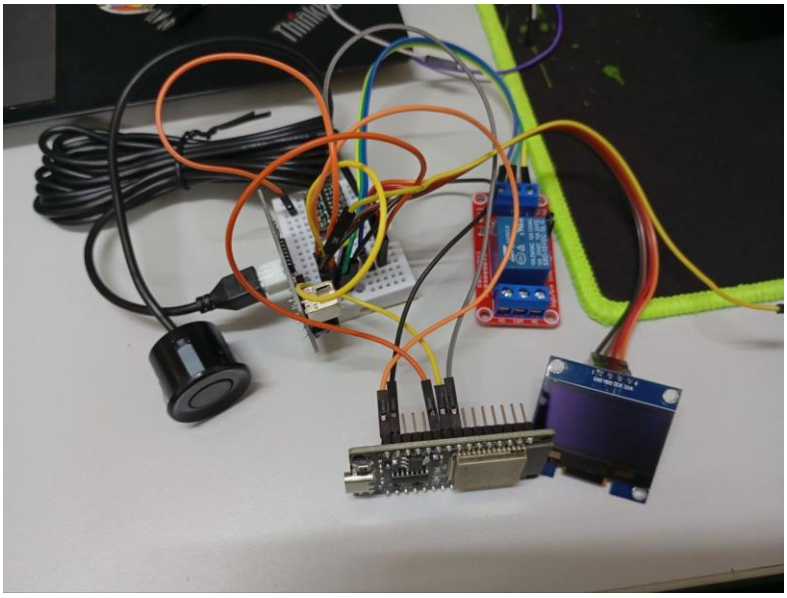
Relay	ESP32
DC+	VIN
DC-	GND
IN	D27



3. Connect OLED Display to ESP32

Pin Connection:

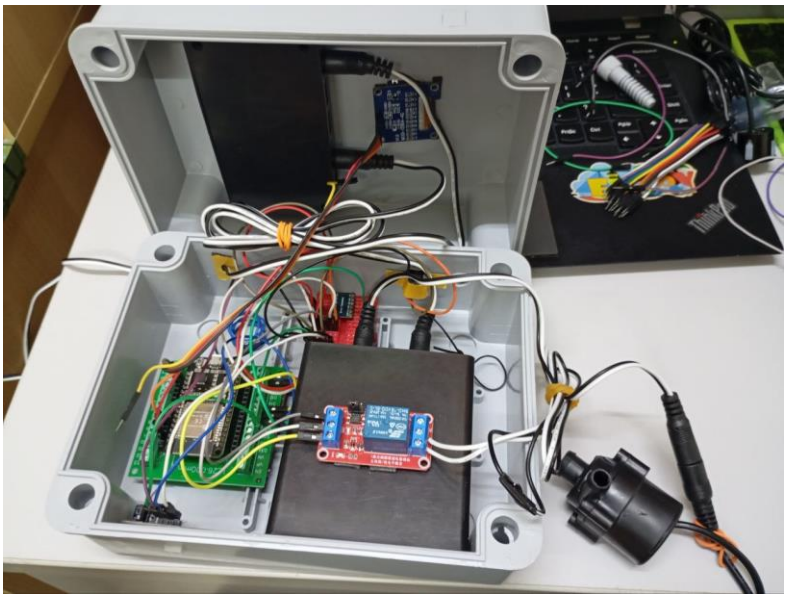
OLED	ESP32
VDD	VIN
GND	GND
SDA	21
SCK	22



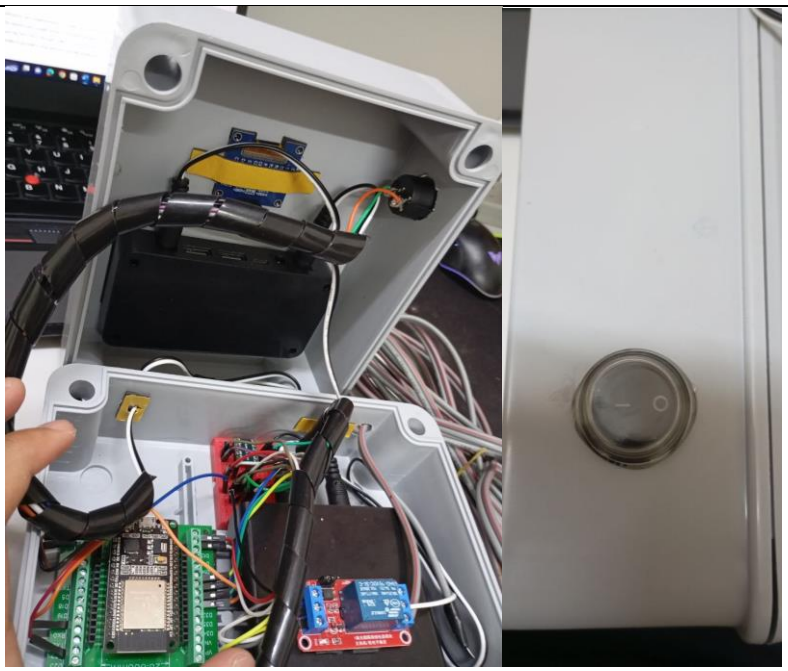
<table>
<tr>
<td>4.</td>
<td>Connect solar panel to its controller then its voltage output connects to battery to store source and supply for pump operation</td>
<td> 
</td>
</tr>
<tr>
<td>5.</td>
<td>Connect Water Pump to Relay switch including 12V Solar power source with battery storage

Pin Connection:

<table>
<tr><th>Relay</th><th>Pump</th></tr>
<tr><td>COM</td><td>+ve first end</td></tr>
<tr><td>NO</td><td>+ve second end</td></tr>
</table>

The +ve connection of water pump is cutted in the middle by relay switch to allow ESP32 control the pump operation and the –ve pump commonly connect to –ve battery</td>
<td>  </td>
</tr>
</table>

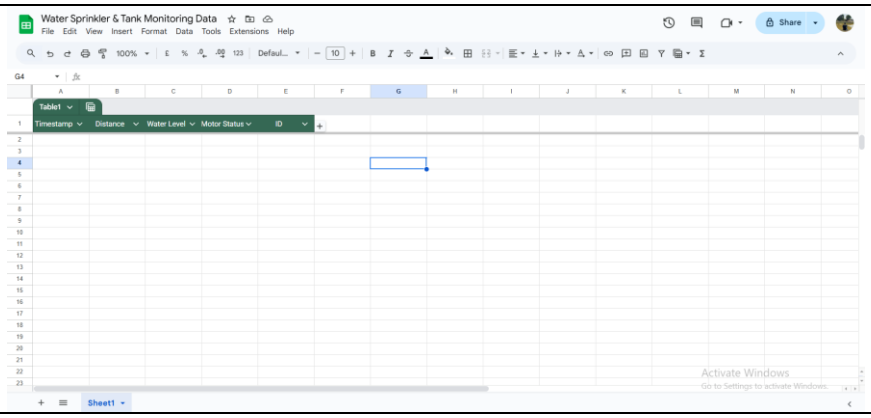
<table>
<tr>
<td>6.</td>
<td>Connect button to ESP32 for bypass the scheduled pump operation

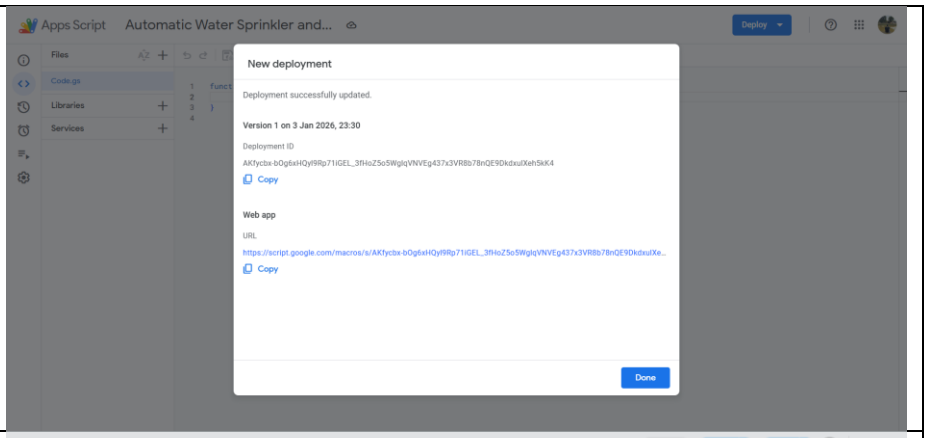
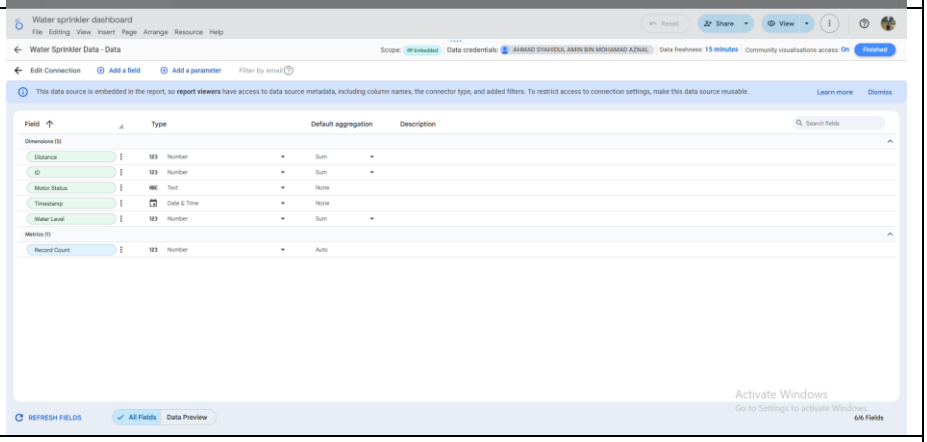
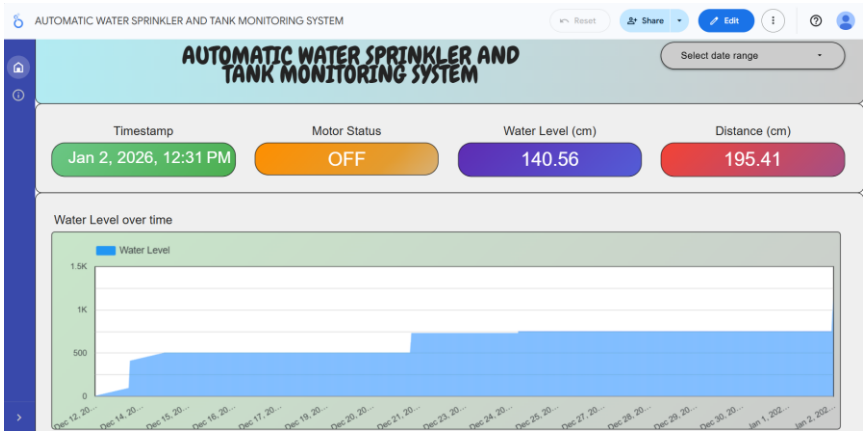
Pin Connection:
<table>
<tr><th>Button</th><th>ESP32</th></tr>
<tr><td>Pin1</td><td>D33</td></tr>
<tr><td>Pin2</td><td>GND</td></tr>
</table>
</td>
<td>  </td>
</tr>
</table>

3.2.3 DATA LOGGING AND VISUALIZATION STEP

Data logging and visualization are implemented to enable real-time monitoring and record system performance. The system uses Google Sheets as an online database to store water level and motor status data sent by the ESP32 through a Wi-Fi connection. The stored data is then visualized using Looker Studio to provide a clear and user-friendly dashboard.

Table 9: Data Logging & Visualization Step

NO	WORK STEP	ATTACHMENT
1.	Create a database structure including Timestamp, Water Level and Motor Status	

2.	Deploy the Google Sheet to obtain the Deployment ID to allow ESP32 send the data	
3.	Link data resource of Google Sheet to the Looker Studio	
4.	Create a visualization chart to display the real-time and historical data of the system	

3.3 PROJECT USABILITY FLOW CHART

This flowchart clearly illustrates the sequential logic, decision points, and looping behavior of the overall system operation.

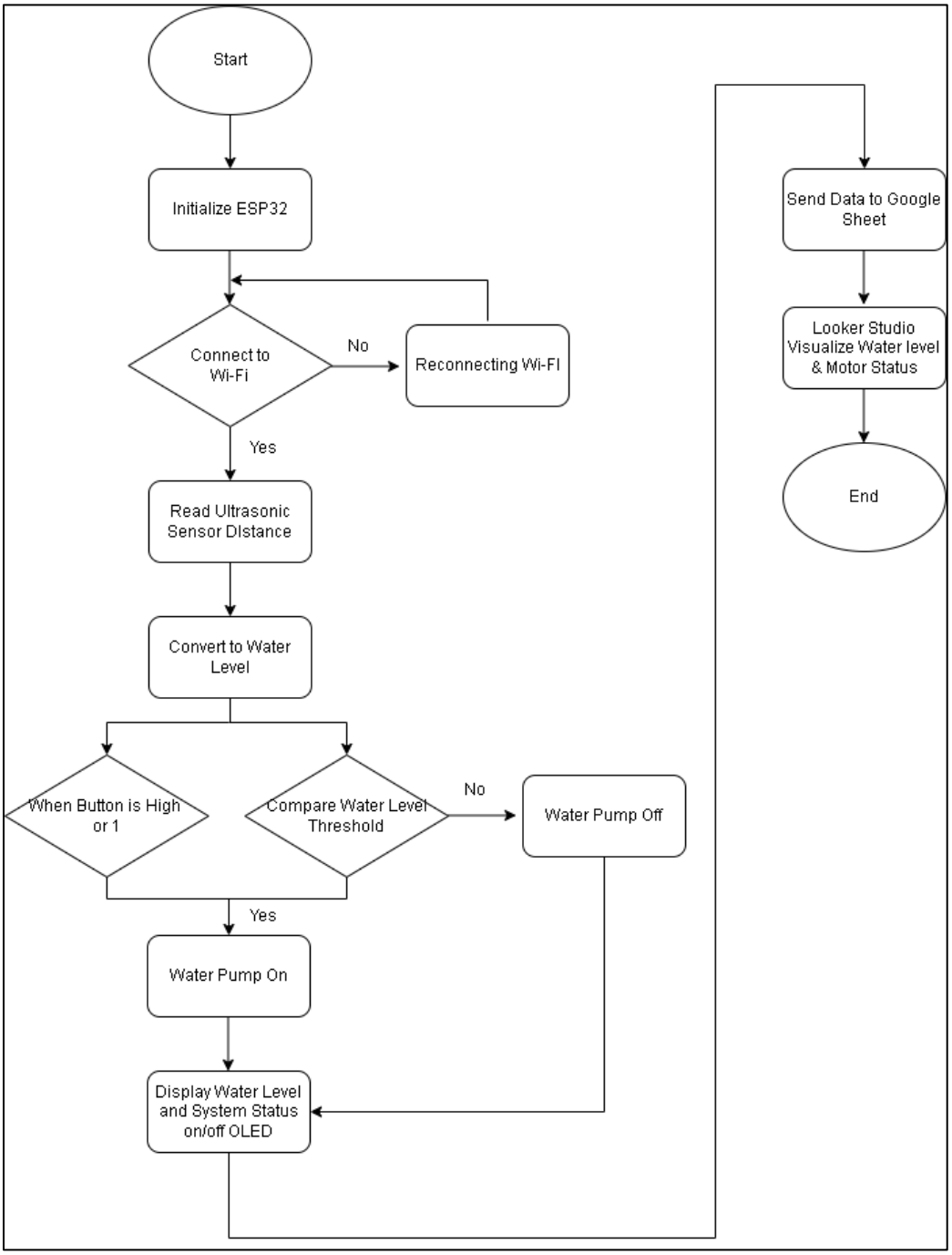


Figure 25: Project Usability Flowchart

3.4 GANTT CHART

Table 10: Gantt Chart

Project Progress Timechart of Automatic Sprinkler Watering System Breakdown by logical phases, duration and member responsibility											
No	Activity	Member	Weeks								Milestone
			week 7	week 8	week 9	week 10	week 11	week 12	week 13	week 14	
Phase 1: Initialization											
1	Component Testing (Sensors, Relays, Board, etc)	SYAMSUL, AMIRUL, AHMAD									Verified functionality of relay module, ESP32 and JSN-SR04T
2	ESP32 GPIO Test & Arduino IDE Setup	AMIRUL									Configured Pins for TRIG (GPIO 5) and ECHO (GPIO 18).
3	Ultrasonic Sensor Reading Test	SYAMSUL									Verified sensor measurement log and accuracy
Phase 2: Integration and Core Functionality											
4	Relay & Pump Control Test	AMIRUL									Tested 12V DC pump activation via relay module.
5	Integration of Sensors + Relay + Pump	AHMAD									Integrated hardware architecture
6	Wi-Fi Connectivity & Google Sheets Logging	SYAMSUL, AMIRUL									Established HTTPS POST communication to google sheet using API Key
7	Sprinkler Schedule Programming (Twice Daily)	SYAMSUL									Programmed automatic cycles (Morning, Evening, Night).
Phase 3: Refinement and Assembly											
8	Safety Logic Implementation (Tank Level Check)	AMIRUL									Added logic to prevent dry-running below 15% level
9	Enclosure Assembly & Outdoor Wiring	SYAMSUL, AMIRUL									Installed enclosure and 3D sensor holder
Phase 4: Deployment and Reporting											
10	Initial Looker Studio Dashboard Setup	AHMAD									Built real-time visualization for water level and pump status
14	Final Outdoor Deployment	AMIRUL									Installed at UMPSA Pekan Landscape Department tank.
15	Final Dashboard & Reporting Setup	SYAMSUL, AHMAD									Completed technical report and cost/benefit analysis.
											Project planning
											Project execution

Embedded Water Sprinkler Timeline

3.5 PROJECT COSTING

Project costing is an important aspect of system development, as it provides an overview of the financial requirements and feasibility of the proposed solution. This section outlines the estimated cost of hardware components used in the ESP32-based automatic water sprinkler and tank monitoring system. The cost analysis is based on current market prices in Malaysia.

Table 11: Project Costing

No	Item	Net Price (RM)	Quantity	Subtotal Price (RM)
1	NodeMCU ESP32 Wi-Fi + Bluetooth	RM 25.16	1	RM 25.16
2	Relay Module 3.3V 5V	RM 3.90	2	RM 7.80
3	ESP32 BASE (Terminal Block) for 30P	RM 8.51	1	RM 8.51
4	Ultrasonic Sensor Waterproof (JSN-SR04T)	RM 26.90	1	RM 26.90
5	Enclosure Box	RM 11.98	1	RM 11.98
6	AC DC Adapter	RM 13.90	1	RM 13.90
7	Logic Level Converter	RM 1.50	1	RM 1.50
8	Female Power Jack	RM 1.20	1	RM 1.20
9	Round Rocker Switch 2 pin	RM 1.29	1	RM1.29
10	Round Rocker Switch 2 pin Cap	RM 0.79	1	RM 0.79
11	3D Print Air Ventilation	RM 30.00	1	RM 30.00
12	Clear PVC Flexible Tube High Duty Water Air Tubing Plastic	RM 3.00 (1 meter	1 x 1.5 meter	RM 4.50
Total Price (after discount)				**RM 133.53**

CHAPTER 4

FINDING AND ANALYSIS

4.1 RESULT & ANALYSIS

The Automatic Water Sprinkler and Tank Monitoring System was successfully implemented and tested at the designated UMPSA Pekan site. Based on the experimental observations, the system achieved stable and reliable operation throughout the testing period.

The whole process was fully automated according to three predefined schedules: morning, evening, and night. With each scheduled interval, the ESP32 microcontroller executed the control algorithm by first verifying the water level inside the tank before activating the pump. This ensured that irrigation was only performed when sufficient water was available.

Throughout operation, the ESP32 maintained a stable connection to the UMPSA Wi-Fi network, allowing sensor data and pump status to be transmitted to Google Sheets at 15-minute intervals. The successful logging of data confirms the reliability of communication and cloud integration components.

Key Result:

When the measured water level dropped below the predefined safety threshold (20 cm), the system successfully prevented activating pump. This dry-run protection mechanism effectively protects the pump from operating without water, reducing the risk of mechanical damage, overheating, and motor failure. The result demonstrates that the safety logic implemented in the system is functional and reliable.

4.2 SYSTEM FUNCTIONAL TESTING

Functional testing was carried out to ensure that each subsystem operated correctly and interacted properly with other components.

Figure 26: System Functionality Testing

4.2.1 ULTRASONIC SENSOR TESTING (JSN-SR04T)

The JSN-SR04T ultrasonic sensor was tested at multiple water levels ranging from empty to full Conditions. The sensor consistently detected distance changes as the water level varied. Initial readings showed minor fluctuations due to water surface movement however, after calibration and averaging, the readings became stable and repeatable.

Table 12: Ultrasonic Sensor Testing

Test No.	Actual Water Level	Measured Distance (cm)	Status
1	50cm	94.7cm	Pass
2	45cm	99.5cm	Pass
3	40cm	104.1cm	Pass

4.2.2 PUMP CONTROL AND RELAY OPERATION TEST

The relay-controlled water pump was tested based on a predefined irrigation schedule consisting of three operating periods: morning, afternoon, and evening. During each scheduled time, the ESP32 controller sent a control signal to the relay module to activate the water pump, provided that the water level in the tank was above the minimum safety threshold.

Table 13: Pump Control and Relay Operation Testing

Test No.	Schedule Time	Water Level	Motor Status	Status
1	07.00 AM	94.7cm	ON	Pass
2	12.00 PM	99.5cm	ON	Pass
3	19.00 Pm	30 cm (Threshold)	OFF	Pass

The test results showed that the pump was successfully activated during all scheduled time slots when sufficient water was available. If the water level dropped below the predefined threshold, the relay remained deactivated even during the scheduled period, preventing dry running of the pump. This confirms that the system prioritizes safety conditions over scheduled operation.

Overall, this test verified that the pump control logic operates reliably based on both time scheduling and real-time water level monitoring, ensuring efficient irrigation while protecting the pump from potential damage.

4.2.3 MANUAL PUMP CONTROL USING PUSH BUTTON TEST

The manual pump control functionality was tested using a physical push button connected to the ESP32 controller. This feature allows the user to activate or deactivate the water pump manually, independent of the predefined irrigation schedule. When the button was pressed, the ESP32 immediately triggered the relay module to turn ON the water pump, and the pump remained active for as long as the button was held. Upon releasing the button, the relay was deactivated and the pump turned OFF. During manual operation, automatic scheduled control was temporarily overridden to prevent conflict between manual and automatic modes.

Table 14: Manual Pump Control Push Button Test

Test No.	Button State	Motor Status	Status
1	Button Pressed	ON	Pass
2	Button Released	OFF	Pass
3	Button Pressed	ON	Pass

This confirms that the manual control feature functions reliably and safely, providing users with direct control while maintaining essential pump protection mechanisms.

4.2.4 OLED DISPLAY FUNCTIONALITY TEST

The OLED display (SH110X) was tested to evaluate its ability to present real-time system information clearly and accurately. The display was configured to show three main parameters: system timestamp, water tank level percentage, and motor (pump) status.

Table 15: OLED Display Functionality Testing

Test No.	Actual Display	OLED Display	Status
1	Time 11:23 Water Level 124.56 Motor Status OFF	Time 11:23:08 Water Level 124.5 Motor Status OFF	PASS
2	Time 14:48 Water Level 122.93 Motor Status OFF	Time 14:40.08 Water Level 122.9 Motor Status OFF	PASS
3	Time 19:00 Water Level 120.74 Motor Status ON	Time 19:00:08 Water Level 120.7 Motor Status ON	PASS

During testing, the timestamp was updated continuously based on the system clock, allowing users to identify the exact time of system operation. The water level percentage displayed on the OLED matched the values obtained from the ultrasonic sensor readings, while the motor status correctly indicated whether the pump was in ON or OFF condition.

4.2.5 PORTABLE SUPPLY ESP32 LONGEVITY TEST

The 5 V circuit of the system, which powers the ESP32 and other low-voltage components, was powered using a **power bank** to make the system fully portable, instead of relying on a fixed 5 V power adapter. During testing, the system was run continuously throughout the day to control the water pump schedule and monitor the water tank level. Even after running all day, the power bank remained nearly fully charged, indicating that the system consumes very low current and that a portable power source is sufficient for long-term operation without frequent recharging. This demonstrates that the low-voltage circuit can reliably operate independently, enhancing the portability and flexibility of the system.

Table 16: Portable Supply ESP32 Longevity Testing

Time Period	System Status		Battery Level
6:00 am – 9:59 am	ESP32	ON	Full bar
	Relay	ON Once (1 min)	
	Ultrasonic	OK	
10:00 am – 1:59 pm	ESP32	ON	Full bar
	Relay	ON Once (1 min)	
	Ultrasonic	OK	
2:00 pm – 5:59 pm	ESP32	ON	Full bar
	Relay	OFF	
	Ultrasonic	OK	
6:00 pm – 8:00 pm	ESP32	ON	Full bar
	Relay	ON Once (1 min)	
	Ultrasonic	OK	

The ESP32 and other low-voltage components were powered using a 5 V power bank to test system portability. The system operated continuously from **6 AM to 8 PM**, controlling the water pump schedule and monitoring the water tank level. After **14 hours** of continuous operation, the power bank remained in full bar or maybe dropping from 100% to 95%. This demonstrates that the low-voltage circuit consumes very little current and can reliably operate on a portable power source for long periods.

4.3 SENSITIVITY SETTINGS

The JSN-SR04T waterproof ultrasonic sensor required calibration to accommodate the physical characteristics of the water tank. Two reference distances were established

Table 17: Sensitivity Setting Test

No	Distance	Water Level	Status
1	120 cm	Full	PASS
2	20 cm	Low or empty	PASS

These values were used to map the measured distance into a corresponding water level percentage. During initial testing, fluctuations in distance readings were observed due to water surface ripples and turbulence generated during pump operation.

The results indicate that the calibrated ultrasonic sensor provides sufficient accuracy for irrigation control and tank monitoring applications, making it suitable for real-world deployment.

4.4 VISUALIZATION RESULT

Here are the results obtained from the local and online visualization of the system. It consists of OLED Display to visualize important information of the system to act as an indicator of our system. The second visualization is the Looker Studio Dashboard to visualize more interactive and more detailed data of the system for historical and analysis.

4.4.1 OLED DISPLAY

The OLED display mounted on to our panel and presents real-time data including

- Timestamp
- Pump operation status (ON/OFF)
- Current water level

This allows on-site personnel to quickly access our system or panel without needing access to our cloud platforms.

Figure 27: OLED Display

4.4.2 LOOKER STUDIO DASHBOARD

For remote monitoring, data logged in Google Sheets is visualized using Looker Studio. The dashboard displays:

- Time-series graphs of water level changes
- Pump operation status real-time and historical
- System activity trends over time (15 minutes interval)

The visualization enables us to analyze historical patterns, detect abnormal behavior such as rapid water depletion, and verify system performance (pump status). This visualization approach significantly improves system transparency and operational awareness.

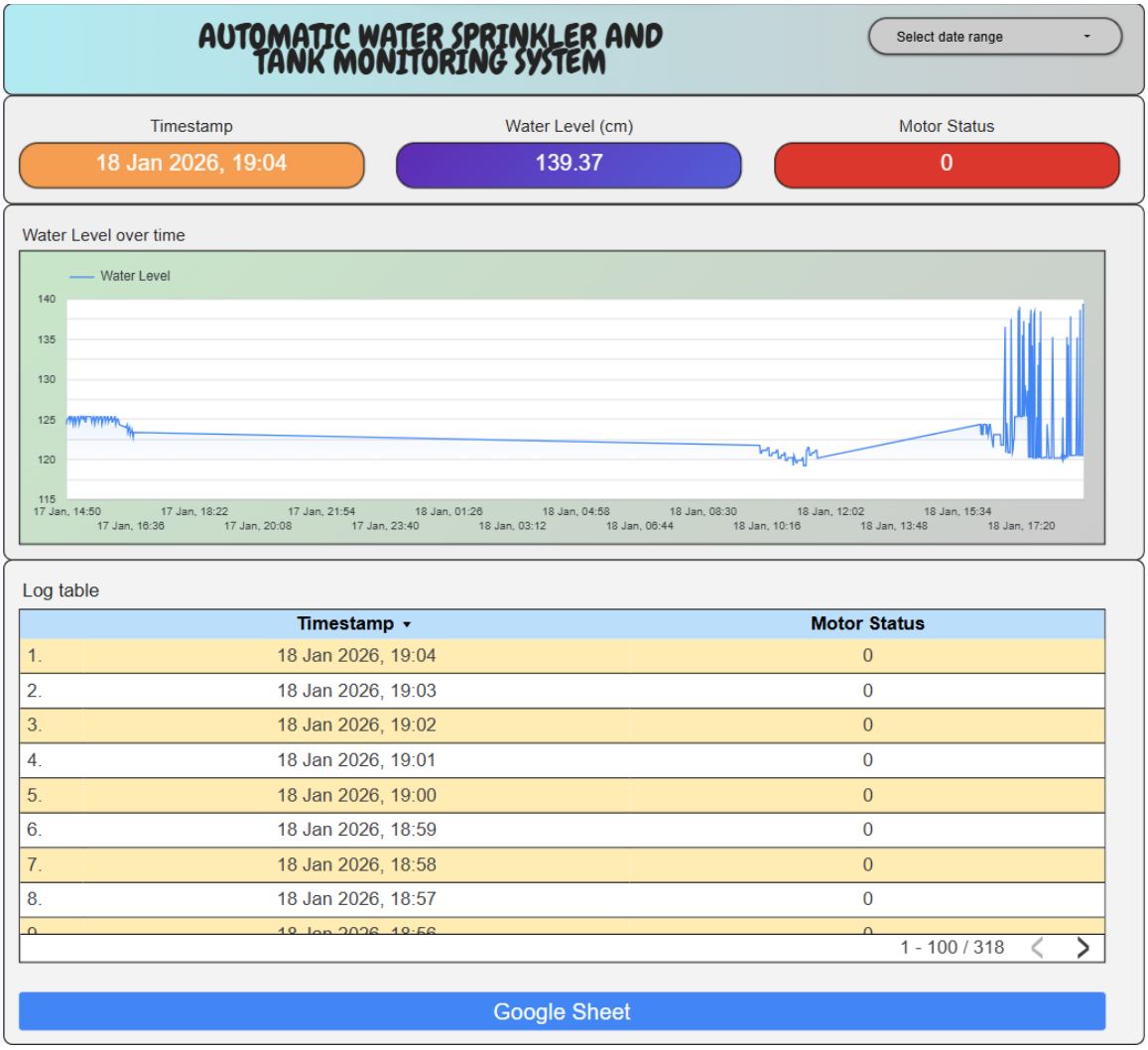


Figure 28: Looker studio Dashboard display

4.5 3D PRINTED COMPONENT DESIGN AND FITMENT TESTING

To ensure seamless integration between the monitoring electronics and the existing water infrastructure, a custom mechanical interface was developed. This section details the transition from the original tank fixture to the specialized 3D-printed housing designed to secure the ultrasonic sensor.

4.5.1 Design and CAD Modelling

The design process began with an analysis of the **original tank opening**. To ensure a watertight and stable fit, the custom component was designed as a flanged "sleeve" that mimics industrial piping standards.

The dimensions were calculated to achieve two primary goals:

- **Acoustic Isolation:** The long cylindrical body acts as a waveguide, ensuring the ultrasonic pulses are directed straight toward the water surface without interference from the tank walls.
- **Debris Protection:** An integrated mesh filter was added to the bottom of the probe to prevent insects or large debris from interfering with the transducer while allowing air to flow freely for distance measurement.

Figure 29: Original Model

Figure 30: Modified Model (3D Modelling)

4.5.2 Fitment Testing and Integration

This stage evaluated the alignment between the printed flange and the tank's mounting points.

The fitment test focused on these key metrics:

Table 18: 3D Modelling Fitment Testing

No	Test	Explanation	Verified
1	Mechanical compatibility	Verify that aligned with the tank’s existing model	Verified
2	Stability	Ensure that sensor remained straight to the water surface avoiding mis signaling or signal multipathing	Verified
3	Cable management	Ensure the top side of sensor wiring is water proofing ensure it remains protected from rain	Verified

Figure 31: Model Assembled on Tank

The 3D-printed modelling (shown in Figure 30) successfully seated into the tank, providing a stable sensor position and waterproof.

4.5.3 Full System Integration and End-to-End deployment

After completing the mechanical fitment and individual subsystem tests, a full system integration and end-to-end functional testing was conducted to evaluate the overall performance of the Automatic Water Sprinkler and Tank Monitoring System under real operating conditions.

In this final configuration, the JSN-SR04T ultrasonic sensor was securely mounted on top of the water tank using the custom 3D-printed housing described in the previous section. This ensured accurate and stable water level measurements while providing adequate protection against environmental exposure. The sensor was directly connected to the control panel via waterproof cabling.

A solar panel was installed near the tank to serve as the primary renewable energy source for the low-voltage monitoring system. The solar panel was connected to a charge controller inside the system panel, which managed battery charging and protected the battery from overcharging and deep discharge. The battery status and solar charging condition were continuously monitored within the panel.

Inside the system control panel, the ESP32 microcontroller, battery, solar charge controller, relay module, and OLED display were fully integrated. The OLED display provided real-time visualization of key system parameters, including system timestamp, current water level, pump operation status (ON/OFF), and power status indicators. This allowed users to easily verify system operation directly from the panel without requiring cloud access.

For pump operation, an external power supply was used through a portable power inverter. This inverter converted the stored electrical energy into the appropriate AC supply required by the water pump. The ESP32 controlled the pump indirectly by switching the relay module, which acted as an electrical isolation interface between the low-voltage control circuit and the high-power pump supply.

During testing, the complete system was operated continuously according to the predefined irrigation schedule programmed in the ESP32 firmware. At each scheduled time, the controller verified the water level before allowing pump activation. If the measured water level exceeded the minimum safety threshold, the relay was energized and the pump was turned ON for the specified duration. If the water level was below the threshold, the pump remained OFF even during scheduled operation.

The end-to-end testing confirmed that all subsystems—including sensing, control logic, power management, visualization, cloud data logging, and pump actuation—functioned correctly as an integrated system. The results demonstrate that the system is capable of autonomous operation with renewable energy support, reliable monitoring, and safe pump control under real-world conditions.

Figure 32: Full system integration setup

4.6 CHALLENGES

Several challenges were encountered during development and deployment:

Table 19: Challenges

No.	Challenge	Description
1.	**Wi-Fi Coverage Limitations**	Some areas got weak Wi-Fi signals, causing us intermittent data transmission. ESP32 won't be able to push the data into the clouds. This requires the use of a high-gain antenna for the ESP32 Wi-Fi coverage. 3

2.	**Signal Noise**	The JSN-SR04T sensor operates at 5V logic levels, while the ESP32 uses 3.3V from ESP32 GPIO pins. Direct connections causing unstable readings as it got not enough power supply. The integration of a bi-directional logic level converter successfully resolved this issue and ensured safe signal translation.
3.	**Waterproofing Constraints**	The environment exposes the risk to electronic components. So, an IP-rated enclosure and proper sealing techniques were applied to ensure the protection of circuitry and all the components.
4.	**ESP32 Overheat**	The ESP32 overheated when the water pump was running because both devices shared the same power supply. The pump drew high current, causing power instability that affected the ESP32. This issue was solved by replacing the ESP32 and separating the power supply. After that, the system operated normally without overheating.

CHAPTER 5

RECOMMENDATION AND SOLUTION

5.1 INTRODUCTION

This chapter discusses potential improvements and future enhancements for the developed system. Although the project achieved its objectives, several upgrades can be implemented to increase system intelligence, scalability, and robustness for campus deployment.

5.2 RECOMMENDATION

5.2.1 Soil Moisture Integration:

Integrating soil moisture sensors would allow irrigation to be based on actual soil conditions rather than fixed schedules alone. This enhancement would further reduce water wastage and improve plant health by delivering water only when necessary.

5.2.2 Mobile App:

Developing a mobile application using platforms such as Flutter or Blynk would allow users to receive real-time notifications, manually override pump operation, and monitor system status remotely.

5.2.3 Solar Power Optimization and Energy Monitoring:

Adding battery voltage and solar charging status monitoring would improve power management and allow predictive maintenance of the energy system.

5.3 INPUT AND IMPROVEMENTS FROM SULAM PROGRAM

The Service Learning Malaysia – University for Society (SULAM) program conducted in collaboration with Kolej Vokasional (KV) Pertanian Chenor provided valuable practical on user driven feedback that contribute to identify potential improvement for our Automatic Water Sprinkler and Tank Monitoring System. Through system demonstrations, technical discussion, and hands on engagement with KV students and instructors, several enhancement opportunities were proposed to improve the system's applicability especially for agricultural use cases.

5.3.1 Integration of Agricultural Sensors (Soil Moisture and pH sensor)

One of the primary improvements suggested during the SULAM program was the integration of additional agricultural sensors, such as soil moisture sensors and pH sensors. Currently, agricultural activities such as fertilizer mixing and irrigation require manual monitoring of soil and water parameters before the fertilization process can begin. By incorporating soil moisture and pH sensing, the system can provide real time environmental data, enabling more accurate in decision making before water and fertilizer application. This enhancement would reduce reliance on manual checks, improve fertilization accuracy, and support more efficient and consistent agricultural practices.

5.3.2 Fully Solar Powered System for Outdoor Deployment

Secondly, feedback from KV participants highlighted the importance of ensuring full energy independence for outdoor agricultural deployments. Since many agricultural are open field locations lack access to a stable electrical power supply, it was recommended that the system be developed as a fully solar powered solution. By utilizing solar panels with appropriate battery storage as the primary power source for both the control system and pump operation, the system can operate continuously without dependence on grid electricity. This improvement enhances system reliability, portability, and suitability for remote and rural environments.

5.3.3 Mobile Application for monitoring and notification

In addition, the SULAM program proposed the development of a mobile application for monitoring and notification purposes. A mobile based platform would allow users to remotely monitor system status, water levels, sensor readings, and pump operation in real time. Furthermore, notification features such as low water level alerts, abnormal sensor readings, or system faults would improve responsiveness and maintenance efficiency. This enhancement would significantly improve user accessibility and system management, especially for agricultural operators managing multiple sites.

Overall, the inputs and feedback obtained through the SULAM program at Kolej Vokasional significantly improve the system's scope beyond basic irrigation automation. The proposed enhancements strengthen the system's relevance to real world agricultural applications, improve operational autonomy, and increase user convenience, ensuring that the developed solution remains scalable, sustainable, and aligned with modern smart agriculture practices.

5.4 INDIVIDUAL ROLES AND RESPONSIBILITES

This project was completed by three group members, each assigned specific roles based on their technical strengths. All members were collectively responsible for system integration, coordination, and preparation of the final project report. Documentation activities were shared among all members to ensure consistency, accuracy, and completeness of the technical report.

The detailed individual responsibilities are as follows:

a) Syamsul Arifin Bin Awi Bowo (VC23047)

Responsible primarily for hardware development and field implementation of the system. Involved in component wiring, relay and pump integration, power system setup, and on-site installation. Actively conducted field testing, troubleshooting, and performance verification of the hardware components. Also contributed to technical documentation related to hardware configuration, testing procedures, and system deployment.

b) **Muhammad Amirul Adly Bin Mohamad Rashidi (VC23053)**

Responsible for software development and system programming. Developed and implemented the ESP32 firmware, including ultrasonic sensor interfacing, pump control logic, scheduling functions, and Wi-Fi communication. Also handled cloud data transmission using Google Apps Script integration. Contributed to system documentation, particularly in software implementation, logic flow, and system functionality explanation.

c) Ahmad Syahidul Amin Bin Mohamad Aznal (VC23054)

Responsible for web-based data visualization and mechanical design. Developed the Looker Studio dashboard for real-time and historical system monitoring. Designed the 3D-printed sensor holder and contributed to mechanical integration and fitment testing. Also participated in system analysis and contributed to documentation related to data visualization, system performance evaluation, and design justification.

5.5 CONCLUSION

In conclusion, this project successfully demonstrated the design and implementation of an IoT-based Automatic Water Sprinkler and Tank Monitoring System suitable for campus applications. The system effectively automates irrigation, monitors water levels in real time, and protects the pump from dry running through intelligent control logic.

The integration of Google Sheets and Looker Studio provides a cost effective, yet professional monitoring platform, enabling data-driven decision-making and long-term performance analysis. Overall, the project supports UMPSA's smart campus and sustainability initiatives by improving water management efficiency, reducing manual labor, and enhancing system reliability.

REFFERENCES

KELETOOL. (2025). pump air solar kebun solar air pump pam oksigen solar solar water pump solar submersible pump solar pam air kebun 100w. Retrieved from shopee.com.my website: https://shopee.com.my/pump-air-solar-kebun-solar-air-pump-pam-oksigen-solar-solar-water-pump-solar-submersible-pump-solar-pam-air-kebun-100w-i.783270835.22281997994?extraParams=%7B%22display_model_id%22%3A69704483833%2C%22model_selection_logic%22%3A3%7D&sp_atk=0c6151be-471c-45b4-99fb-28317e17bd7e&xptdk=0c6151be-471c-45b4-99fb-28317e17bd7e

Kumar Kunal, Md. Azhar Husain, & N Srinivasan. (2019, October). Smart Irrigation and Tank Monitoring System. Retrieved from Researchgate website: https://www.researchgate.net/publication/336566575_Smart_Irrigation_and_Tank_Monitoring_System

MANORSHI. (2025). JSN-SR04T Ultrasonic Module Ranging Module Distance. Retrieved from Manorshi.com website: https://www.manorshi.com/JSN-SR04T-Ultrasonic-Module-Ranging-Module-Distance-Sensor-pd46813559.html

Manorshi. (2025). Ultrasonic Sensor Module JSN-SR04T-3.0. Retrieved from https://www.manorshi.com/ website: https://jlrorwxhjiilll5q.ldycdn.com/JSN-SR04T-3.0-aidnqBpoKliRljSlqnqkilqj.pdf

MYBOTIC. (2020). Single Channel 5V Relay Module. Retrieved from Mybotic.com.my website: https://www.mybotic.com.my/products/Single-Channel-5V-Relay-Module/478

Petru Livinti, Marius V Tivlea, & Popa Sorin Eugen. (2023, April 23). Water Level Control and Monitoring System in a Tank Made with Arduino Uno and NodeMCU ESP8266 Development Boards. https://doi.org/10.20944/preprints202304.0735.v1

pursuit1.my. (2025). 800L/H 5m DC 12V 24V Solar Water Heater Brushless Motor Circulation Water Pump. Retrieved from shopee.com.my website: https://shopee.com.my/product/296623352/4148736935?gads_t_sig=VTJGc2RHVmtYMTlxTFVSVVRrdENkWVp3RFo3Mkw5czd4Z0hzdEF1WVFia3NtOEg0aFRnZGk2dm1QWHJ0OVRsaXFtU3BkMEREdEd4M3I2ZzQrQm94WEovQUc5bCtkUVQveVFJWHlibG5BeDhPQkRnK3ZLUXhVZzM1U204L1pQOVc1elVSNXpEMTk2VHhtUDZuVnd1Z21BPT0&gad_source=1&gad_campaignid=22986531517&gbraid=0AAAAADPpYTLPMFVimNZKLGWR5tBoZcMQ7&gclid=Cj0KCQiA9t3KBhCQARIsAJOcR7y7hI2mjTvdAOsKqN11QH6hfwxlp6mjqCOOu-QWT3WqX2KajuofGNUaArZcEALw_wcB

Simple Circuit. (2025, July 29). Interfacing Arduino Board with SH1107 OLED Display in I2C Mode. Retrieved from simple-circuit.com website: https://simple-circuit.com/interfacing-arduino-sh1107-oled-display-i2c-mode/

TechMaker. (2025). 4 Channel Logic Level Converter Bi-directional Module 5V - 3.3V. Retrieved from Techmakers.com.my website: https://techmakers.com.my/electronic-module/4channel-logic-level-converter-bi-directional-module-5v-3-3v?sort=rating&order=ASC&limit=75

Wilson, J. (2020, December 16). ESP32 Pinout, Datasheet, Features & Applications - The Engineering Projects. Retrieved from www.theengineeringprojects.com website: https://www.theengineeringprojects.com/2020/12/esp32-pinout-datasheet-features-applications.html

APPENDIX

Code Arduino IDE

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>

// ====================== WIFI ======================
const char* ssid = "MAABMR";
const char* password = "Yoi121203";

// ============= Google Script Web App URL ==========
String scriptURL =
        "https://script.google.com/macros/s/AKfycbwmdCnWfVO1yniji1tp6pGKi_Sa4aGA9p9QJETFSDhC
        6eUOy13_b40vt1VNPqEN4XZT/exec";

// =================== Ultrasonic Pins ==============
#define TRIG 26
#define ECHO 25

// =================== Relay Pin ====================
#define RELAY 27

// =================== Manual Button ================
#define BUTTON_PIN 33   // Button to GND (INPUT_PULLUP)

// ================= OLED SH110X ====================
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SH1106G display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ================ NTP Time (Malaysia UTC+8) ========
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", 8 * 3600, 60000);

// ============= Tank Settings ======================
float tankHeight = 145.0;
float lowWaterCutoff = 30.0;

// ============= Watering Flags =====================
bool waterMorning = false;
bool waterEvening = false;
bool waterNight = false;

// ============= Motor Status =======================
bool manualMode = false;
String motorStatusText = "OFF";  // OLED & Serial
```

```
int motorStatusValue = 0;       // Google Sheets (1=ON, 0=OFF)

// ================= Timers ========================
unsigned long lastSheetMillis = 0;
const unsigned long sheetInterval = 60000;  // 60s Google Sheet interval

// AUTO watering timers
unsigned long autoStartMillis = 0;
unsigned long autoDuration = 60000;  // 1 minute watering
bool autoRunning = false;

// Next scheduled AUTO watering times (HH,MM)
struct AutoSchedule {
  int hour;
  int minute;
  bool done;
};
AutoSchedule morning = {17, 10, false};
AutoSchedule evening = {17, 20, false};
AutoSchedule night   = {17, 30, false};

// ================================================
void setup() {
  Serial.begin(115200);

  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  pinMode(RELAY, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);

  digitalWrite(RELAY, LOW);

  // ================= OLED INIT =================
  Wire.begin(21, 22);
  if (!display.begin(0x3C, true)) {
    Serial.println("OLED not detected");
    while (1);
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SH110X_WHITE);
  display.setCursor(0, 0);
  display.println("Water System");
  display.println("Initializing...");
  display.display();

  // ================= WIFI ======================
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
```

```
  }
  Serial.println("\nWiFi Connected!");

  // ================= NTP ======================
  timeClient.begin();
}

// ================ Ultrasonic ====================
float readDistance() {
  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  long duration = pulseIn(ECHO, HIGH, 30000);
  if (duration == 0) return tankHeight;

  return duration * 0.034 / 2;
}

// ================ OLED ========================
void updateOLED(String timeStr, float waterLevel) {
  display.clearDisplay();

  display.setCursor(0, 0);
  display.println("Water Tank Monitor");
  display.drawLine(0, 10, 127, 10, SH110X_WHITE);

  display.setCursor(0, 15);
  display.print("Time: ");
  display.println(timeStr);

  display.setCursor(0, 30);
  display.print("Motor: ");
  display.println(motorStatusText);

  display.setCursor(0, 45);
  display.print("Level: ");
  display.print(waterLevel, 1);
  display.println(" cm");

  display.display();
}

// ================ Google Sheets =================
void sendToSheets(String timestamp, float distance, float waterLevel) {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin(scriptURL);
    http.addHeader("Content-Type", "application/json");

    String json = "{";
```

```
  json += "\"Timestamp\":\"" + timestamp + "\",";
  json += "\"Distance\":" + String(distance, 2) + ",";
  json += "\"Water Level\":" + String(waterLevel, 2) + ",";
  json += "\"Motor Status\":" + String(motorStatusValue);
  json += "}";

  int httpResponseCode = http.POST(json);
  Serial.println("Sent to Google Sheets:");
  Serial.println(json);
  Serial.println("HTTP Response: " + String(httpResponseCode));
  http.end();
 }
}

// ===================== LOOP =====================
void loop() {
 timeClient.update();

 int hour = timeClient.getHours();
 int minute = timeClient.getMinutes();
 String timestamp = timeClient.getFormattedTime();

 float distance = readDistance();
 float waterLevel = tankHeight - distance;
 if (waterLevel < 0) waterLevel = 0;

 // ================= MANUAL CONTROL =================
 manualMode = (digitalRead(BUTTON_PIN) == LOW);
 if (manualMode) {
  digitalWrite(RELAY, HIGH);
  motorStatusText = "MANUAL ON";
  motorStatusValue = 1;
  autoRunning = false; // stop AUTO if manual pressed
 } else {
  // If AUTO running, keep it ON
  if (autoRunning) {
   digitalWrite(RELAY, HIGH);
   motorStatusText = "AUTO ON";
   motorStatusValue = 1;
  } else {
   digitalWrite(RELAY, LOW);
   motorStatusText = "OFF";
   motorStatusValue = 0;
  }
 }

 // ================= AUTO SCHEDULE (non-blocking) =================
 if (!manualMode && waterLevel > lowWaterCutoff) {
  // Morning
  if (!morning.done && hour == morning.hour && minute == morning.minute) {
   autoRunning = true;
   autoStartMillis = millis();
   morning.done = true;
```

```
  }
  // Evening
  if (!evening.done && hour == evening.hour && minute == evening.minute) {
    autoRunning = true;
    autoStartMillis = millis();
    evening.done = true;
  }
  // Night
  if (!night.done && hour == night.hour && minute == night.minute) {
    autoRunning = true;
    autoStartMillis = millis();
    night.done = true;
  }
}

// AUTO running timer
if (autoRunning) {
  if (millis() - autoStartMillis >= autoDuration) {
    autoRunning = false;
    digitalWrite(RELAY, LOW);
    motorStatusText = "OFF";
    motorStatusValue = 0;
  }
}

// ============== RESET FLAGS AT MIDNIGHT ==========
if (hour == 0 && minute == 0) {
  morning.done = false;
  evening.done = false;
  night.done = false;
}

// ================= DISPLAY =================
updateOLED(timestamp, waterLevel);

// ================= SERIAL ==================
Serial.printf(
  "Time: %s | Distance: %.2f cm | Water Level: %.2f cm | Motor: %s (%d)\n",
  timestamp.c_str(),
  distance,
  waterLevel,
  motorStatusText.c_str(),
  motorStatusValue
);

// ================= GOOGLE SHEETS (Non-blocking 60s) =================
unsigned long currentMillis = millis();
if (currentMillis - lastSheetMillis >= sheetInterval) {
  sendToSheets(timestamp, distance, waterLevel);
  lastSheetMillis = currentMillis;
}
delay(1000);  // loop refresh for button / OLED updates
}
```

Code App Script

```
function doGet(e){
 return ContentService.createTextOutput("Web App is running and ready for POST requests.");
}

function doPost(e) {
 try {
  var ss = SpreadsheetApp.getActiveSpreadsheet();
  var sheetName = "Data";

  // Check if sheet exists, create if not
  var sheet = ss.getSheetByName(sheetName);
  if (!sheet) {
   sheet = ss.insertSheet(sheetName);
  }

  // ====== AUTO-HEADER PROTECTION ======
  var headers = ["Timestamp","Distance","Water Level","Motor Status","ID"];

  // If header row empty OR not match → rewrite header
  var firstRow = sheet.getRange(1, 1, 1, headers.length).getValues()[0];
  var headerMissing = false;

  for (var i = 0; i < headers.length; i++) {
   if (firstRow[i] !== headers[i]) {
    headerMissing = true;
    break;
   }
  }

  if (headerMissing) {
   sheet.getRange(1, 1, 1, headers.length).setValues([headers]);
  }

  // Parse JSON from ESP32
  var data = JSON.parse(e.postData.contents);

  // ===== AUTO-INCREMENT ID =====
  var lastRow = sheet.getLastRow();
  var newID = 0;

  if (lastRow > 1) {
   var lastID = sheet.getRange(lastRow, 5).getValue();
   newID = Number(lastID) + 1;
  }

  // Insert new data row
  sheet.appendRow([
   new Date(),             // Timestamp
   data.Distance,          // Distance
   data["Water Level"],    // Water Level
   data["Motor Status"],   // Motor Status
```

```
    newID                   // Auto ID
  ]);

  return ContentService.createTextOutput("OK");

 } catch(err) {
  return ContentService.createTextOutput("Error: " + err);
 }
}
```

APPENDIX

Link video explanation full system

Automatic Water Sprinkler and Tank Monitoring System

Link Youtube :- https://youtu.be/Le2bylP9K3I?si=LCYah106wKKn48SZ